

১. Mediator Pattern কী? (সোজাভাবে)

- ধরুন অনেক objects আছে, যারা একে অপরের সাথে communicate করতে চায়।
- **Before:** objects সরাসরি একে অপরকে call করে → messy এবং tightly coupled।
- **Mediator:** একটি central object সব communication handle করে → objects একে অপরকে জানে না।

Analogy:

- School → Teacher (Mediator) → students communicate via teacher, direct talk নয়

২. Before Mediator (Messy code)

ধরা যাক, Chat app এ 3 users: Alice, Bob, Charlie

```
class User {
    private String name;

    public User(String name) { this.name = name; }

    // Directly sending message to another user
    public void sendMessage(User to, String message) {
        System.out.println(this.name + " sends to " + to.getName() +
            ": " + message);
    }

    public String getName() { return name; }
}

public class MainBefore {
    public static void main(String[] args) {
        User alice = new User("Alice");
        User bob = new User("Bob");
        User charlie = new User("Charlie");
```

```

        alice.sendMessage(bob, "Hi Bob!");
        bob.sendMessage(charlie, "Hello Charlie!");
        charlie.sendMessage(alice, "Hey Alice!");
    }
}

```

Problem:

- Alice, Bob, Charlie একেকটি একে অপরকে জানে → tightly coupled
- New user add করলে সব class modify করতে হবে
- Communication messy

Code থেকে চিনার উপায়:

- Object → direct call other object methods
- Communication logic scattered

৩. After Mediator (Solved with pattern)

Step 1: Mediator Interface

```

interface ChatMediator {
    void sendMessage(String message, User sender);
    void addUser(User user);
}

```

Step 2: Concrete Mediator

```

import java.util.ArrayList;
import java.util.List;

class ChatRoom implements ChatMediator {
    private List<User> users = new ArrayList<>();
}

```

```

@Override
public void addUser(User user) {
    users.add(user);
}

@Override
public void sendMessage(String message, User sender) {
    for(User u : users) {
        if(u != sender) {
            u.receiveMessage(message, sender);
        }
    }
}
}

```

Step 3: Colleague (User)

```

abstract class User {
    protected String name;
    protected ChatMediator mediator;

    public User(String name, ChatMediator mediator) {
        this.name = name;
        this.mediator = mediator;
    }

    public abstract void sendMessage(String message);
    public abstract void receiveMessage(String message, User sender);
}

class ChatUser extends User {
    public ChatUser(String name, ChatMediator mediator) {
        super(name, mediator);
    }

    @Override
    public void sendMessage(String message) {

```

```

        System.out.println(this.name + " sends: " + message);
        mediator.sendMessage(message, this);
    }

    @Override
    public void receiveMessage(String message, User sender) {
        System.out.println(this.name + " received from " + sender.name
+ ": " + message);
    }
}

```

Step 4: Client

```

public class MainAfter {
    public static void main(String[] args) {
        ChatMediator chatRoom = new ChatRoom();

        User alice = new ChatUser("Alice", chatRoom);
        User bob = new ChatUser("Bob", chatRoom);
        User charlie = new ChatUser("Charlie", chatRoom);

        chatRoom.addUser(alice);
        chatRoom.addUser(bob);
        chatRoom.addUser(charlie);

        alice.sendMessage("Hi everyone!");
        bob.sendMessage("Hello Alice!");
        charlie.sendMessage("Hey guys!");
    }
}

```

Output After:

```

Alice sends: Hi everyone!
Bob received from Alice: Hi everyone!
Charlie received from Alice: Hi everyone!
Bob sends: Hello Alice!
Alice received from Bob: Hello Alice!

```

Charlie received from Bob: Hello Alice!
Charlie sends: Hey guys!
Alice received from Charlie: Hey guys!
Bob received from Charlie: Hey guys!

8. Before vs After সহজভাবে

Feature	Before	After Mediator
Coupling	Tight → users know each other	Loose → users only know mediator
Communication logic	Scattered	Centralized in mediator
Flexibility	Low	High → add new user easily
Maintainability	Low	High
Extensibility	Hard → new user add difficult	Easy → just register new user

Key insight:

- **Before:** objects direct call → messy
 - **After:** Mediator handles communication → decoupled, maintainable, scalable
-

যদি চান, আমি Mediator + Observer + State + Template + Strategy সবগুলোর **Before & After** একসাথে **table + short example** বানিয়ে দেখাতে পারি, যাতে শুধু কোড দেখে সব pattern চেনা যায়।

আপনি কি সেটা চাইবেন?

আপনি বলেছেন:

mane sbi msg ta pabe?

ChatGPT বলেছে:

না, **Mediator Pattern** এ যেটা হয় সেটা হলো “**central hub**” বা **mediator handle** করে, কিন্তু প্রতি **message** সবাই পাবেন না, বরাবর mediator decide করে কাকে পাঠানো হবে।

উদাহরণ দিয়ে ব্যাখ্যা করি:

Case 1: Broadcast (সবাই পাবে)

@Override

```
public void sendMessage(String message, User sender) {  
    for(User u : users) {  
        if(u != sender) { // sender skip  
            u.receiveMessage(message, sender);  
        }  
    }  
}
```

- এখানে Alice message পাঠালে **Bob** এবং **Charlie** দুজনই receive করবে
- Sender (Alice) নিজে পাবেনা

✓ এই ক্ষেত্রে message → সবকেই পৌঁছে

Case 2: Targeted (কেবল নির্দিষ্ট ব্যক্তি পাবেন)

@Override

```
public void sendMessage(String message, User sender, User receiver) {  
    receiver.receiveMessage(message, sender);  
}
```

- এখানে Alice Bob কে message পাঠালে শুধুমাত্র **Bob** পাবেন
- Charlie পাবেন না

✓ Mediator decide করে **who receives the message**